Rajarata University of Sri Lanka

Department of Computing

ICT 1305 – Data Structures
Laboratory 05 – Linked list in C
Time: 120 Minutes

Objective:

By the end of this lab, students should be able to:

- Understand the structure and operations of doubly linked lists.
- Implement basic operations including insertion, deletion, and traversal.
- Develop confidence in pointer manipulation with bidirectional node references.

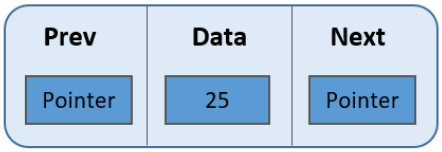
Doubly Linked List

A **Doubly Linked List (DLL)** is a linear data structure where each node contains a data part and two pointers:

- A pointer to the **next node**.
- A pointer to the **previous node**.

This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.

A doubly linked list is a more complex data structure than a singly linked list.



Activity 1: Define a Node and Create an Empty DLL

Each node contains data and two pointers: one pointing to the next node and another pointing to the previous node.

Task: **Write a program to define a DLL node structure and create an empty DLL.**

```
#include <stdio.h>

#include <stdlib.h>



// Define the structure for a doubly linked list node
struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {

   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

   // Check if memory allocation was successful
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      return NULL;
   }


   newNode->data = data;
   newNode->next = NULL;
   newNode->prev = NULL;

   return newNode;
}

int main() {
   // Create an empty doubly linked list
   struct Node* head = NULL;

   printf("Empty doubly linked list created successfully.\n");
```

```
    printf("Head pointer: %p\n", (void*)head);

    return 0;
}
```

Activity 2: Insert a Node at the Beginning

This function inserts a new node at the beginning of the doubly linked list. It handles both empty list and non-empty list cases, updating the previous pointer of the old first node.

Task: **Implement a function to insert a node at the beginning of the DLL.**

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a doubly linked list node
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

// Function to insert a node at the beginning of the DLL
struct Node* insertAtBeginning(struct Node* head, int data) {
    // Create a new node
    struct Node* newNode = createNode(data);
    if (newNode == NULL) {
        return head;  // Return unchanged head if allocation failed
    }

    // If the list is empty, new node becomes the head
```

```
    if (head == NULL) {
        return newNode;
    }

    // Link the new node to the current head
    newNode->next = head;
    head->prev = newNode;

    // New node becomes the new head
    return newNode;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;  // Initialize empty list

    // Insert nodes at the beginning
    head = insertAtBeginning(head, 30);
    head = insertAtBeginning(head, 20);
    head = insertAtBeginning(head, 10);

    printf("After inserting 30, 20, 10 at beginning:\n");
    displayForward(head);

    return 0;
}
```

Activity 3: Insert a Node at the End

This function inserts a new node at the end of the doubly linked list. It traverses to the last node and updates the links appropriately.

Task: **Implement a function to insert a node at the end of the DLL.**

```
#include <stdio.h>
```

```
#include <stdlib.h>

// Define the structure for a doubly linked list node
struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      return NULL;
   }
   newNode->data = data;
   newNode->next = NULL;
   newNode->prev = NULL;
   return newNode;
}

// Function to insert a node at the end of the DLL
struct Node* insertAtEnd(struct Node* head, int data) {

   struct Node* newNode = createNode(data);
   if (newNode == NULL) {
      return head;  // Return unchanged head if allocation failed
   }

   if (head == NULL) {
      return newNode;
   }

   // Traverse to the last node
   struct Node* current = head;
   while (current->next != NULL) {
      current = current->next;
   }

   // Link the new node to the last node
   current->next = newNode;
   newNode->prev = current;
```

```
    return head;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;  // Initialize empty list

    // Insert nodes at the end
    head = insertAtEnd(head, 10);
    head = insertAtEnd(head, 20);
    head = insertAtEnd(head, 30);

    printf("After inserting 10, 20, 30 at end:\n");
    displayForward(head);

    return 0;
}
```

Activity 4: Insert a Node at a Given Position

This function inserts a new node at a specified position in the doubly linked list. It handles edge cases like inserting at the beginning or end.

Task: **Implement a function to insert a node at a specified position (1-based index).**

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a doubly linked list node
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

```
// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      return NULL;
   }
   newNode->data = data;
   newNode->next = NULL;
   newNode->prev = NULL;
   return newNode;
}

// Function to insert a node at a given position (1-based index)
struct Node* insertAtPosition(struct Node* head, int data, int position) {
   // Check for invalid position
   if (position < 1) {
      printf("Invalid position! Position should be >= 1\n");
      return head;
   }

   // Create a new node
   struct Node* newNode = createNode(data);
   if (newNode == NULL) {
      return head;
   }

   // If inserting at position 1 (beginning)
   if (position == 1) {
      if (head != NULL) {
         newNode->next = head;
         head->prev = newNode;
      }
      return newNode;  // New node becomes the head
   }

   // Traverse to the desired position
   struct Node* current = head;
   for (int i = 1; i < position && current != NULL; i++) {
      current = current->next;
   }
```

```
    // If position is beyond the list length, insert at the end
    if (current == NULL) {
        // Find the last node
        current = head;
        while (current->next != NULL) {
            current = current->next;
        }
        // Insert at the end
        current->next = newNode;
        newNode->prev = current;
        printf("Position beyond list length. Inserted at end.\n");
    } else {
        // Insert at the specified position
        newNode->next = current;
        newNode->prev = current->prev;

        if (current->prev != NULL) {
            current->prev->next = newNode;
        }
        current->prev = newNode;
    }

    return head;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Insert nodes at different positions
    head = insertAtPosition(head, 20, 1);
    head = insertAtPosition(head, 10, 1);
    head = insertAtPosition(head, 30, 3);
    head = insertAtPosition(head, 25, 3);
```

```
    printf("After inserting nodes at different positions:\n");
    displayForward(head);

    return 0;
}
```

Activity 5: Delete a Node from the Beginning

This function deletes the first node from the doubly linked list and updates the head pointer appropriately.

Task: **Implement a function to delete the first node in the DLL.**

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a doubly linked list node
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

// Function to delete the first node from the DLL
struct Node* deleteFromBeginning(struct Node* head) {
    // Check if the list is empty
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return head;
    }
```

```
    // Store the node to be deleted
    struct Node* nodeToDelete = head;

    // Update head to point to the next node
    head = head->next;

    // If the new head is not NULL, update its prev pointer
    if (head != NULL) {
        head->prev = NULL;
    }

    // Print the data of the deleted node
    printf("Deleted node with data: %d\n", nodeToDelete->data);

    // Free the memory of the deleted node
    free(nodeToDelete);

    return head;  // Return the new head
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Create a sample list: 10 -> 20 -> 30
    head = createNode(10);
    head->next = createNode(20);
    head->next->prev = head;
    head->next->next = createNode(30);
```

```
    head->next->next->prev = head->next;

    printf("Original list:\n");
    displayForward(head);

    // Delete from beginning
    head = deleteFromBeginning(head);
    printf("After deleting from beginning:\n");
    displayForward(head);

    // Delete from beginning again
    head = deleteFromBeginning(head);
    printf("After deleting from beginning again:\n");
    displayForward(head);

    return 0;
}
```

Activity 6: Delete a Node from the End

This function deletes the last node from the doubly linked list by traversing to the end and updating the links.

Task: **Implement a function to delete the last node in the DLL.**

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a doubly linked list node
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
```

```
    newNode->prev = NULL;
    return newNode;
}

// Function to delete the last node from the DLL
struct Node* deleteFromEnd(struct Node* head) {
    // Check if the list is empty
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return head;
    }

    // If there's only one node
    if (head->next == NULL) {
        printf("Deleted node with data: %d\n", head->data);
        free(head);
        return NULL;
    }

    // Traverse to the last node
    struct Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }

    // Update the second-to-last node's next pointer
    current->prev->next = NULL;

    // Print the data of the deleted node
    printf("Deleted node with data: %d\n", current->data);

    // Free the memory of the deleted node
    free(current);

    return head;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    if (head == NULL) {
        printf("List is empty\n");
        return;
```

```
    }
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Create a sample list: 10 -> 20 -> 30
    head = createNode(10);
    head->next = createNode(20);
    head->next->prev = head;
    head->next->next = createNode(30);
    head->next->next->prev = head->next;

    printf("Original list:\n");
    displayForward(head);

    // Delete from end
    head = deleteFromEnd(head);
    printf("After deleting from end:\n");
    displayForward(head);

    // Delete from end again
    head = deleteFromEnd(head);
    printf("After deleting from end again:\n");
    displayForward(head);

    return 0;
}
```

Activity 7: Delete a Node at a Given Position

This function deletes a node at a specified position in the doubly linked list, handling various edge cases.

Task: **Implement a function to delete a node at a given position.**

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Define the structure for a doubly linked list node
struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      return NULL;
   }
   newNode->data = data;
   newNode->next = NULL;
   newNode->prev = NULL;
   return newNode;
}

// Function to delete a node at a given position (1-based index)
struct Node* deleteAtPosition(struct Node* head, int position) {
   // Check for invalid position
   if (position < 1) {
      printf("Invalid position! Position should be >= 1\n");
      return head;
   }

   // Check if the list is empty
   if (head == NULL) {
      printf("List is empty! Cannot delete.\n");
      return head;
   }

   // If deleting the first node
   if (position == 1) {
      struct Node* nodeToDelete = head;
      head = head->next;

      if (head != NULL) {
         head->prev = NULL;
      }
```

```
        printf("Deleted node with data: %d\n", nodeToDelete->data);
        free(nodeToDelete);
        return head;
    }

    // Traverse to the desired position
    struct Node* current = head;
    for (int i = 1; i < position && current != NULL; i++) {
        current = current->next;
    }

    // If position is beyond the list length
    if (current == NULL) {
        printf("Position beyond list length! Cannot delete.\n");
        return head;
    }

    // Update the links of neighboring nodes
    if (current->prev != NULL) {
        current->prev->next = current->next;
    }

    if (current->next != NULL) {
        current->next->prev = current->prev;
    }

    // Print the data of the deleted node
    printf("Deleted node with data: %d\n", current->data);

    // Free the memory of the deleted node
    free(current);

    return head;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
    printf("Forward traversal: ");
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    while (head != NULL) {
```

```
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    // Create a sample list: 10 -> 20 -> 30 -> 40
    head = createNode(10);
    head->next = createNode(20);
    head->next->prev = head;
    head->next->next = createNode(30);
    head->next->next->prev = head->next;
    head->next->next->next = createNode(40);
    head->next->next->next->prev = head->next->next;

    printf("Original list:\n");
    displayForward(head);

    // Delete at position 3
    head = deleteAtPosition(head, 3);
    printf("After deleting at position 3:\n");
    displayForward(head);

    // Delete at position 1
    head = deleteAtPosition(head, 1);
    printf("After deleting at position 1:\n");
    displayForward(head);

    return 0;
}
```

Activity 8: Display the List (Forward and Backward)

Task: **Write two functions: one for traversing and displaying the DLL in forward direction, and one for displaying it in reverse direction.**

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Define the structure for a doubly linked list node
struct Node {
   int data;
   struct Node* next;
   struct Node* prev;
};

// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      return NULL;
   }
   newNode->data = data;
   newNode->next = NULL;
   newNode->prev = NULL;
   return newNode;
}

// Function to display the list in forward direction
void displayForward(struct Node* head) {
   printf("Forward traversal: ");
   if (head == NULL) {
      printf("List is empty\n");
      return;
   }

   // Traverse from head to tail
   while (head != NULL) {
      printf("%d", head->data);
      if (head->next != NULL) {
         printf(" <-> ");
      }
      head = head->next;
   }
   printf(" -> NULL\n");
}

// Function to display the list in backward direction
void displayBackward(struct Node* head) {
   printf("Backward traversal: ");
   if (head == NULL) {
```

```
        printf("List is empty\n");
        return;
    }

    // First, traverse to the last node
    struct Node* tail = head;
    while (tail->next != NULL) {
        tail = tail->next;
    }

    // Now traverse from tail to head
    printf("NULL <- ");
    while (tail != NULL) {
        printf("%d", tail->data);
        if (tail->prev != NULL) {
            printf(" <-> ");
        }
        tail = tail->prev;
    }
    printf("\n");
}

// Function to insert a node at the end (helper function)
struct Node* insertAtEnd(struct Node* head, int data) {
    struct Node* newNode = createNode(data);
    if (newNode == NULL) {
        return head;
    }

    if (head == NULL) {
        return newNode;
    }

    struct Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }

    current->next = newNode;
    newNode->prev = current;

    return head;
}
```

```
// Function to free the entire list
void freeList(struct Node* head) {
   struct Node* temp;
   while (head != NULL) {
      temp = head;
      head = head->next;
      free(temp);
   }
   printf("Memory freed successfully.\n");
}

int main() {
   struct Node* head = NULL;

   // Create a sample doubly linked list: 10 <-> 20 <-> 30 <-> 40
   head = insertAtEnd(head, 10);
   head = insertAtEnd(head, 20);
   head = insertAtEnd(head, 30);
   head = insertAtEnd(head, 40);

   printf("Doubly Linked List Traversal:\n");
   printf("========\n");

   // Display in forward direction
   displayForward(head);

   // Display in backward direction
   displayBackward(head);

   // Free the memory
   freeList(head);

   return 0;
}
```

Summary Points

• **Doubly Linked Lists** have two pointers per node: next and prev, allowing bidirectional traversal.

• **Memory Management** is crucial - always free allocated memory to prevent memory leaks.

• **Pointer Updates** require careful attention to maintain list integrity during insertion/deletion.

• **Bidirectional Traversal** is a key advantage over singly linked lists.

Key Differences from Singly Linked Lists

Aspect	**Singly Linked List**	**Doubly Linked List**
Memory per node	Less (only next pointer)	More (next + prev pointers)
Traversal	Forward only	Forward and backward
Deletion	Requires previous node reference	Can delete with node reference only
Insertion	Simpler pointer updates	More complex pointer updates
Use cases	When memory is limited	When bidirectional traversal is needed

- End -